BACK MATTER

Appendix B: RFCs as precedent

Three protocol specifications that have outlived implementations written against them, and what an organization can borrow from the way they were written.

Every claim in this book about specifications outliving implementations has already been tested, for longer than any of this book's examples have existed, by the documents that describe how computers talk to each other. This appendix looks at three of them: RFC 791, RFC 793, and RFC 7230. None of the three cares what language implements it. Each has outlived implementations written against it. This book borrows that pattern, and the borrowing deserves a clear look before the comparison to organizational specifications.

What an RFC refuses to say

An **RFC**, short for Request for Comments, is a numbered document describing how a piece of internet infrastructure should behave, published today through the Internet Engineering Task Force (IETF). The name undersells it. The first RFCs, starting in 1969, really were working notes soliciting comment. What the series became is something closer to law: a body of specifications that most of the internet's traffic still runs on, decades later, without anyone needing to ask permission to implement one.

An RFC that defines a protocol reads like a use case crossed with a domain model. It names the actors: a client, a server, sometimes an intermediary. It states preconditions and postconditions for every exchange between them. It enumerates the valid states a connection can be in and the transitions allowed between them. It specifies what a compliant implementation must do when something goes wrong. What it does not do, ever, is name a programming language, a data structure, or a vendor.

That omission is deliberate, and it is the whole point. An RFC describes what a protocol does. It leaves how any particular system builds that behavior entirely to whoever is implementing it. A specification that tried to pin down implementation would not travel between a mainframe and a phone; naming an implementation choice is exactly what a specification cannot survive doing, if it wants to outlive the technology fashionable the year it was written.

Three specifications, outliving everything built against them

RFC 791 defines the Internet Protocol: how a packet finds its way from one machine to another across networks neither machine controls. **RFC 793** defines the Transmission Control Protocol built on top of it: how two machines agree to open a reliable connection, keep it in order, and close it cleanly. Both were published in September 1981. They standardized work underway through the 1970s, and together they are the reason "TCP/IP" is spoken as one phrase. **RFC 7230**, published in 2014, is one part of a multi-document revision of HTTP, the protocol a browser uses to ask a server for a page.

None of the three names a technology. RFC 7230 says a server responds to a request with a status line, a set of headers, and an optional body, in a specified order, under specified conditions. It does not say which operating system runs the server, which language parses the request, or which company built either one. A server written in C in the 1990s and a server written in a language that did not exist yet when RFC 793 was published still open a TCP connection the same way, because both were built against the same document.

> **What survived, what didn't**
>
> The implementations written against RFC 791 in the early 1980s are gone: retired hardware, discontinued operating systems, companies that no longer exist. RFC 791 is still the specification a new network stack is written against today. The document outlasted every machine that ever ran code built from it.

How a protocol changes without a central rewrite

Protocols evolve, and the RFC series has a mechanism for that: a new RFC supersedes or amends an old one, and implementations update to match. TLS, short for Transport Layer Security, the protocol that secures most web traffic, went through versions 1.0, 1.1, 1.2, and 1.3 across roughly two decades. Four versions, four separate specifications. The protocol evolved through versioned documents rather than through a coordinated edit to every server that speaks it.

What that example does and doesn't show needs a precise reading. Implementations updated to support each new TLS version because the specification told them what the new version required, not because a single unedited codebase silently absorbed every change. Server software was patched, libraries were rewritten, and some old implementations were retired outright rather than upgraded. What stayed constant was the destination: every implementation, however it got there, was aiming at the same published document. That is a more modest claim than "nothing was hand-edited," and it is the one this book is willing to defend.

NOTE

The comparison this appendix draws between a domain model and a protocol, and between a business rule and a protocol constraint, is this book's own analogy. It is not a claim that internet standards bodies or software architects treat organizational specifications as RFCs today. Appendix C works the analogy through in full; here it is stated once, plainly, as an interpretation rather than an established practice.

The mapping this book draws

Line up a protocol specification next to the four-layer specification this book builds across Chapters 2 through 5, and the shapes match closely enough to be useful, even though nobody designed one to resemble the other.

A **domain model** plays the role an RFC gives its message format: both name the things a system reasons about and the relationships between them, before either says a word about storage or transport. A **use case** does the same job a protocol's state machine does, laying out a sequence of steps between named actors, a defined starting point, a defined successful outcome, and the branches that apply when something deviates from the main path. And a **business rule** reads like a protocol constraint. Both state something that must hold no matter which implementation is running, the kind of statement an RFC writes as a "MUST".

The parallel is not exact, and this book does not claim it is. A protocol coordinates strangers who will never meet and have every incentive to interoperate; an organization's specification coordinates colleagues who already share an employer and could, in principle, just ask each other. But the same discipline justifies the parallel in both places: write down what must be true, leave open how any particular system makes it true, and the specification keeps working long after the system that first satisfied it is gone.

Why most organizations never adopted this discipline

If separating what from how has worked this well for fifty years of internet infrastructure, the obvious question is why it stayed at the internet's edge instead of moving inside the organizations that build on top of it. A few reasons are worth naming, though the reading is this book's own.

Software engineering culture rewards shipped features, and an RFC-style specification looks, to a team under deadline pressure, like paperwork standing between them and the thing they're actually paid to build. The feedback loop is part of it too: the IETF can afford to spend years getting a protocol right because thousands of implementers depend on getting it right once. A single product team rarely feels that pressure from the other direction; the cost of an undocumented rule doesn't show up until the one engineer who remembers it leaves, and by then the bill has been separated from the decision that created it. Most organizations also have no equivalent of a standards process. There's no body that reviews a domain model the way a working group reviews a draft RFC, no version history a new hire can read to see how a rule changed and why.

Building that discipline inside an organization means creating what the internet's standards process already supplies: an owner for the specification, and a habit of reviewing every change before it ships. A specification has to become a document someone owns and updates, the way an editor owns a standard, instead of a comment left to decay in the code. Adopting the model costs time in the early cycles. The payoff compounds later, the way it did for the internet: it shows up on the ten-thousandth exchange, handled by an implementation whose authors never met anyone who wrote the original specification.